

VERIQO -- TRUST AUTHORITY MODEL & FORMAL INVARIANT CONTRACT

Response to Pekerjaan_Besar_yang_harus_ditutup_sampai_selesai.docx -- the reviewer's instruction to stop auditing function-by-function and instead raise five audit rounds' worth of findings into (1) a formal VERIQO Trust Authority Model, (2) a formal invariant list as a security contract, (3) a distributed replay / snapshot-forgery adversarial pass, and (4) a concurrency semantic-correctness audit ("race-free != state-machine-safe under concurrent transitions").

Everything below is grounded in this repository's real, compiling, tested code as it stands today -- every type, function, and error sentinel named is a real identifier you can grep for; every invariant cites the actual test function(s) that prove it; every gap this round found was fixed, with a regression test proven via break-test-fix-restore (temporarily disable the fix, confirm the new test fails with the exact expected diagnostic, restore the fix, confirm it passes again).

PART A -- THE VERIQO TRUST AUTHORITY MODEL

A.1 WHY A UNIFYING MODEL, NOT A SINGLE GENERIC TYPE

VERIQO has no one AuthorityObject type that every kind of fact passes through. Five years of incremental hardening (five audit rounds, in this engagement alone) produced four independently-evolved subsystems, each with its own concrete vocabulary:

Subsystem	Package	What it governs
---	---	---
Evidence Manifest	pkg/evidence/manifest	Chain-of-custody and finalization of one physical/digital evidence artifact
Evidence Record	pkg/insurance/evidence	Verification status and usage rights of one evidence item within a case
Hypothesis	pkg/insurance/causation	Causal-claim status derived from supporting/contradicting evidence references
Finding	pkg/insurance/finding + pkg/insurance/cre	A well-evidenced candidate conclusion, gated into an AuthorizedFinding

Inventing a fifth, generic type that all four funnel through would be architecture theater -- it would not run, and it would not be what the real code enforces. Instead, this section defines the ladder as an abstraction: seven rungs that describe the *shape* every one of these subsystems' own lifecycle takes, in its own vocabulary, and a mapping table showing exactly where each subsystem's real states land on it -- including being honest about which rungs a given subsystem does not implement, and why.

A.2 THE CANONICAL LADDER

UNTRUSTED
|
REGISTERED
|
VERIFIED
|
DERIVED
|

AUTHORIZED

|

FINALIZED

|

IMMUTABLE

UNTRUSTED -- Data has crossed the system boundary (a caller's struct literal, an HTTP body, a JSON blob about to be unmarshaled) but has not yet been handed to any constructor this repository controls. Nothing about it is trusted; it is not even a value type the rest of the system recognizes as "in the system" yet.

REGISTERED -- The item has been given a real, addressable identity by a real constructor (manifest.RegisterDraft, evidence.New + Registry.Submit, provenance.Registry.Register, causation.HypothesisSet.Add), but nothing about its *content* has been independently checked yet. Every authority-bearing field is reset to its honest, fail-closed starting value at this rung, regardless of what the caller supplied -- this is precisely INV-001 (Part B).

VERIFIED -- An independent check has been run against the item's own claims: a custody event's ContentHash matches the manifest's own SHA256 (hasCustodyActionBoundToContent); a provenance.Entry's trust was granted via a real GrantTrust call carrying a policy (and, for an evidence provider, an attestation) reference; a Finding passed VerifyFindingAgainstHypothesis and VerifyFindingProvenance. Verification answers "is this claim internally consistent and attributable", not yet "what do we conclude from it".

DERIVED -- A value is *computed* from already-verified inputs, never asserted by a caller. evidence.DeriveStatus(Strength), causation.computeStatus(Hypothesis, *DependencyGraph), and finding.Evaluate are the three real derivation functions in this repository; each is the *only* place its own output field is ever written.

AUTHORIZED -- A governed act has explicitly sanctioned use of the now-verified-and-derived item: evidence.Registry.SetRights (only when authorityID names an entity whose trust was really granted) and cre.Authorize / cre.AuthorizeGrounded (sealing an AuthorizedFinding whose fields are unexported and unconstructable from outside the cre package) are the two real authorization gates.

FINALIZED -- The item's state is locked and a cryptographic commitment is computed over its final, frozen content: manifest.Registry.Advance(..., StateFinalized, ...) computes ManifestHash over the manifest as of that instant and refuses every further direct mutation (ErrFinalizedIsImmutable). AuthorizedFinding is finalized *at construction* -- AuthorizationHash is computed once, inside Authorize, and the type has no mutator at all.

IMMUTABLE -- The finalized record survives forever, exactly as recorded, queryable and independently re-verifiable, and can only be superseded (a new version created alongside it) or revoked (marked no-longer-authoritative) -- never edited in place. manifest.Registry.Supersede and evidence.Registry.MarkSuperseded are the two real supersession mechanisms; both append a new fact rather than rewriting an old one.

A.3 PER-RUNG CONTRACT (GENERIC)

Rung	Who can create it	Evidence required	Cryptographic proof	Permitted transition	Forbidden transition	Derived or asserted	Survives serialization	Survives replay	Survives replication	Revocable	Supersedable
UNTRUSTED	anyone / anything	none	none	-> REGISTERED, via a real constructor	direct promotion to any later rung	asserted (unchecked)	n/a -- not yet in the system	n/a	n/a	n/a	n/a
REGISTERED	the item's own constructor only, with authority fields forced to fail-closed defaults	a real identity (non-empty ID, known enum values)	none yet	-> VERIFIED skipping straight to AUTHORIZED/FINALIZED	default	yes (INV-006)	yes (INV-007)	yes (INV-009)	n/a (nothing to revoke)	n/a	
VERIFIED	the specific verification function for that item (never the caller)	content-hash-bound attestation, or a real trust grant	a check that independently recomputes/re-derives and compares	-> DERIVED asserting VERIFIED with no matching evidence (INV-004/005)	asserted that verification ran; the *result* is a fact, not a guess	yes	yes	yes	yes (trust can be revoked -- RevokeTrust)	n/a	
DERIVED	the one authoritative derivation function for that field	the verified inputs the derivation reads	the derivation function itself is the "proof" -- deterministic and pure	-> AUTHORIZED any direct write to the derived field from outside the derivation function (INV-002)	derived	yes	yes	yes	yes -- re-running derivation on new inputs supersedes the old value	implicit (re-derivation)	
AUTHORIZED	a named authority whose trust was independently granted	a real, attributed grant recorded elsewhere (GrantTrust) or a passed verification gate (Authorize)	the grant's own policy/attestation refs, or AuthorizationHash	-> FINALIZED authorizing without a real, attributed grant (INV-010)	asserted, but only by a checked authority	yes	yes	yes	yes	n/a (a later authorization supersedes)	
FINALIZED	the sole state-transition function for that type, and only when every prerequisite holds	every prerequisite for every prior rung, chained	a hash computed over the frozen content at this instant	-> IMMUTABLE (via Supersede, never via in-place edit)	any direct field mutation after this point (INV-003)	derived (the hash) over asserted-but-verified content	yes -- and the hash must independently re-verify (INV-006/008)	yes -- replay must reach byte-identical hash (INV-007)	yes -- independent nodes computing over identical input converge (INV-009)	no (a FINALIZED item is never itself "revoked" -- see Supersede)	yes, via Supersede/MarkSuperseded
IMMUTABLE	n/a -- this rung describes a property of the historical record, not a new construction	the full original evidentiary chain, permanently retained	the original FINALIZED hash, forever re-verifiable	queryable forever	rewriting or deleting the historical record	n/a	yes	yes	yes	its *successor* can be marked current; the record itself never disappears	is what Supersede/MarkSuperseded produce

A.4 MAPPING THE FOUR REAL SUBSYSTEMS ONTO THE LADDER

This is the honest part: no subsystem implements all seven rungs with a literal one-to-one field. Each implements the subset the reviewer's own four completed audit rounds actually hardened.

EVIDENCE MANIFEST (PKG/EVIDENCE/MANIFEST)

Ladder rung	Manifest's own vocabulary	Real mechanism
UNTRUSTED	a Manifest struct literal, or JSON about to be unmarshaled	nothing controls it yet
REGISTERED	StateDraft (v1)	RegisterDraft unconditionally forces State=DRAFT, Version=1, clears ManifestHash/ManifestSignature, regardless of caller input
VERIFIED	StateIngested / StateIntegrityAssessed / StateProvenanceComplete	transitionPrerequisiteLocked requires a content-hash-bound RECEIVED/HASHED custody event and a recorded AcquisitionRecord for each respective transition

DERIVED *(not separately named -- verification and derivation are the same step here: each transition's prerequisite check IS the derivation of "is this manifest ready")*
transitionPrerequisiteLocked

AUTHORIZED StateReadyForFinalization requires a content-hash-bound REVIEWED custody event

FINALIZED StateFinalized Advance computes ManifestHash over the frozen manifest; refuses any further mutation via ErrFinalizedIsImmutable

IMMUTABLE StateSuperseded (the retired version) + the new version at StateDraft
Supersede -- requires parent.State == StateFinalized; the old version's ManifestHash is never rewritten

EVIDENCE RECORD (PKG/INSURANCE/EVIDENCE)

Ladder rung Record's own vocabulary Real mechanism
--- --- ---

UNTRUSTED a Record struct literal, or JSON about to be unmarshaled nothing controls it yet

REGISTERED StatusUnverified, RightsUnknownPendingContract Registry.Submit
unconditionally resets Status, Strength, Rights, CorrectionSuperseded, SupersededBy to their honest defaults, regardless of caller input

VERIFIED *(rights authority)* SetRights requires authorityID to name a provenance.Entry with TrustGranted == true, itself only settable by GrantTrust

DERIVED Status (post VerifyStatus) DeriveStatus(Strength) is the *only* function that ever computes Status; VerifyStatus stores its result

AUTHORIZED Rights (post SetRights) as above

FINALIZED *(no separate FINALIZED rung -- a Record does not "lock"; its correction path is supersession directly)* --

IMMUTABLE CorrectionSuperseded=true, SupersededBy=<id> MarkSuperseded -- requires non-empty actor/reason, refuses re-supersession (ErrAlreadySuperseded) and an illegitimate successor (ErrIllegitimateSuccessor); the original record and its content-addressed ID are never deleted or rewritten

HYPOTHESIS (PKG/INSURANCE/CAUSATION)

Ladder rung Hypothesis's own vocabulary Real mechanism
--- --- ---

UNTRUSTED a Hypothesis struct literal nothing controls it yet

REGISTERED StatusUnproven HypothesisSet.Add (fixed this round -- see Part C)
unconditionally forces Status=StatusUnproven, regardless of caller input

VERIFIED *(evidence IDs are references, not independently re-verified at this layer -- see A.5)* AddSupportingEvidence/AddContradictingEvidence

DERIVED StatusSupported / StatusPartiallySupported / StatusContradicted / StatusInsufficientEvidence computeStatus/DeriveStatus, driven only by RecomputeStatuses and the Add*Evidence methods -- never settable directly

AUTHORIZED *(a Hypothesis is not itself authorized for use -- that happens one layer up, at Finding)* --

FINALIZED / IMMUTABLE *(a HypothesisSet is a live working model, not a finalized artifact -- it has no lock/supersede step of its own)* --

FINDING (PKG/INSURANCE/FINDING + PKG/INSURANCE/CRE)

Ladder rung Finding's own vocabulary Real mechanism
--- --- ---

UNTRUSTED a Finding struct literal nothing controls it yet

REGISTERED StatusCandidate Finding.Status is documented as "derived-only... never set directly by a caller with the intention that it stick"

VERIFIED *(pre-Authorize)* VerifyFindingAgainstHypothesis, VerifyFindingProvenance

DERIVED StatusFinding Evaluate is the sole writer

AUTHORIZED cre.AuthorizedFinding Authorize / AuthorizeGrounded -- unexported fields

```

make external construction a compile error, not merely a runtime check
  FINALIZED    *(construction-time -- AuthorizedFinding has no separate finalize step; it
is born finalized)*    AuthorizationHash computed once, inside Authorize
  IMMUTABLE    *(no supersession mechanism exists for a Finding today -- a superseding
conclusion is expressed as a NEW AuthorizedFinding citing updated evidence, not an edit)*
--

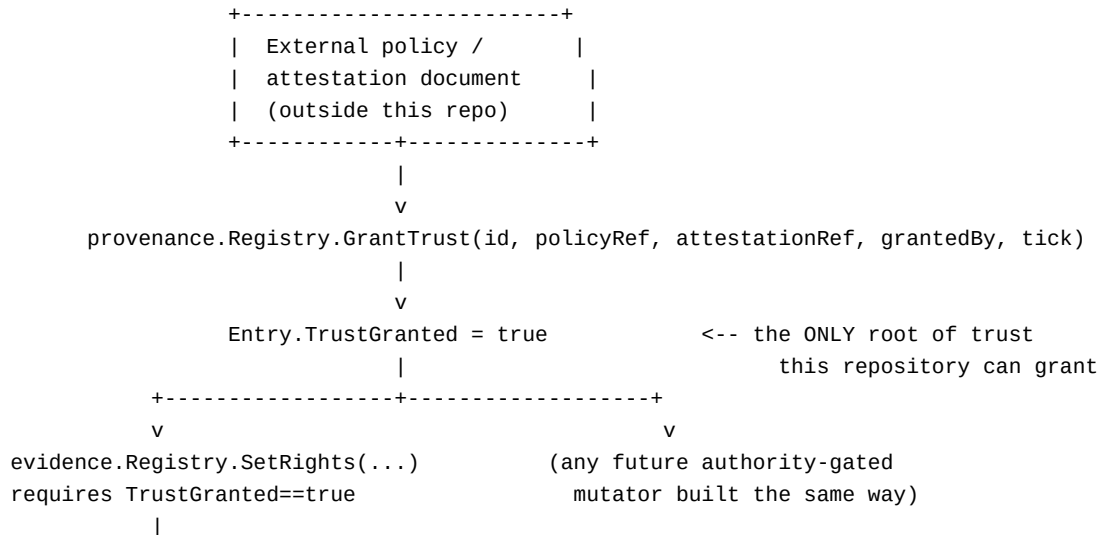
```

A.5 WHERE THE LADDER IS HONESTLY INCOMPLETE

- Hypothesis's "VERIFIED" rung is weak by design. SupportingEvidence/ContradictingEvidence are plain string evidence-ID references; addEvidence does not check the referenced ID resolves to a real, grounded, FINALIZED manifest. This is intentional layering, not an oversight -- cre.AuthorizeGrounded's own doc comment says exactly this: "a reference is just a string ID... nothing in Authorize alone confirms that ID names evidence that actually exists." Grounding is deliberately deferred to the Finding layer (AuthorizeGrounded, which requires every cited ID to resolve to a FINALIZED, hash-verified manifest). A Hypothesis.Status of StatusSupported is therefore a true statement about *the evidence references on file*, not yet a claim that those references are grounded -- exactly as finding.Finding is a "well-evidenced candidate conclusion, not a verdict" until it clears AuthorizeGrounded.
- Manifest has no single "DERIVED" rung because every one of its transitions is gated by the same mechanism (transitionPrerequisiteLocked) that both verifies AND effectively derives readiness in one step; splitting it into two named rungs would not correspond to two different functions in the real code.
- Evidence Record and Finding have no FINALIZED lock. Correction happens by direct supersession (Record) or by simply not authorizing a stale Finding again (Finding) -- there is no ErrFinalizedIsImmutable equivalent for either type, because neither one's design ever promised in-place immutability the way Manifest's ManifestHash does.

This is stated plainly rather than invented: a model claiming uniform coverage across all four subsystems would be fiction. What is real is that every subsystem separately, provably, closes the same category of authority-bypass gap -- which is exactly what Part B's invariant list formalizes.

A.6 TRUST AUTHORITY GRAPH (ROOT OF TRUST, ALL FOUR SUBSYSTEMS)



v
Record.Rights = <authorized state>

manifest.Registry.RecordCustodyEvent(actor, action, contentHash)
|
v

transitionPrerequisiteLocked requires a content-hash-BOUND event
|
v

Advance(... StateFinalized) --computes--> ManifestHash
|
v

VerifyManifestHash independently re-derives and compares <-- verification,
not trust

finding.Evaluate (derivation) --> cre.Authorize / AuthorizeGrounded
requires: VerifyFindingAgainstHypothesis (real hypothesis in a real HypothesisSet)
VerifyFindingProvenance (real InferenceTrace(s))
[Grounded variant] every cited evidence ID resolves to a
FINALIZED, hash-verified manifest.Manifest

|
v
cre.AuthorizedFinding{unexported fields} <-- unconstructable outside cre

causation.HypothesisSet.Add forces Status=StatusUnproven
|
v

AddSupportingEvidence / AddContradictingEvidence / RecomputeStatuses
|
v

computeStatus (pure function of recorded evidence references)

No path in this graph lets a caller reach an authoritative end state (TrustGranted=true, an authorized Rights state, StateFinalized, an AuthorizedFinding, or a derived Status/StatusSupported) without passing through the one function this repository designates as that value's sole writer. Every arrow above is a real function call in the current codebase; there is no arrow this document invented.

PART B -- FORMAL INVARIANT LIST (INV-001 .. INV-010): VERIQ0'S SECURITY CONTRACT

Each invariant below states the reviewer's own wording, the real mechanism that enforces it, and the real test(s) that prove it holds -- including the tests this round added. An invariant with no test citation would be a promise, not a contract; none of the ten are stated without one.

INV-001 -- NO CALLER MAY DIRECTLY CONSTRUCT AN AUTHORITATIVE STATE.

Mechanism: every authority-bearing field is either (a) forced to a fail-closed default at the sole construction/registration entry point regardless of caller input, or (b) held in an unexported field unconstructable from outside its owning package.

- manifest.RegisterDraft forces State=DRAFT, clears ManifestHash/ManifestSignature.
- evidence.Registry.Submit resets Status, Strength, Rights, CorrectionSuperseded, SupersededBy.
- causation.HypothesisSet.Add forces Status=StatusUnproven (this round's fix -- see Part C.1).
- cre.AuthorizedFinding's fields are all unexported; the zero value is the only value obtainable without calling Authorize.

Proven by: TestRegisterDraftStartsAtVersion1,
TestSubmitResetsAuthorityBearingFields,
TestJSONDeserializedRecordCannotManufactureAuthority,
TestJSONDeserializedManifestCannotManufactureAuthority,
TestForgedSnapshotStateFieldCannotBeRestored (this round),
TestHypothesisSetAddNeverTrustsACallerAssertedSupportedStatus (this round).

INV-002 -- AUTHORITATIVE STATE MUST BE DERIVED FROM VERIFIABLE EVIDENCE OR AN AUTHORIZED TRANSITION.

Mechanism: evidence.DeriveStatus is the sole computer of Status; causation.computeStatus/DeriveStatus is the sole computer of Hypothesis.Status; finding.Evaluate is the sole computer of Finding.Status; manifest.Advance's transitionPrerequisiteLocked requires a real, recorded custody event or field before allowing each state transition.

Proven by: TestVerifyStatusDerivesFromStrength (evidence package),
TestAdvanceRefusesIngestedWithNoCustodyEvent,
TestAdvanceRefusesIntegrityAssessedWithoutHashedCustodyEvent,
TestAdvanceRefusesProvenanceCompleteWithoutAcquisitionRecord,
TestAdvanceRefusesReadyForFinalizationWithoutReviewedCustodyEvent,
TestAdvanceRefusesFinalizationWithoutClassification,
TestRecomputeStatusesWithDependencyGraph (causation package).

INV-003 -- FINALIZED EVIDENCE CANNOT BE MUTATED WITHOUT A GOVERNED SUPERSESSION/REVOCATION TRANSITION.

Mechanism: manifest.Advance refuses any transition once cur.State == StateFinalized (ErrFinalizedIsImmutable); Supersede requires the parent to already be StateFinalized (ErrParentNotFinalized otherwise); evidence.MarkSuperseded refuses a second supersession of the same record (ErrAlreadySuperseded) and an illegitimate successor (ErrIllegitimateSuccessor).

Proven by: TestFinalizedManifestIsImmutable,
TestSupersedeRefusesAnUnfinalizedParent,
TestSupersedeCreatesNewVersionWithoutRewritingHistory,
TestConcurrentDoubleFinalizeHasExactlyOneWinner (this round -- proves the guard holds even under concurrent contention, not just sequentially),
TestConcurrentFinalizeAndSupersedeNeverSupersedesAnUnfinalizedParent (this round).

INV-004 -- A VALID HASH DOES NOT ESTABLISH PROVENANCE BY ITSELF.

Mechanism: hasCustodyActionBoundToContent requires a HASHED/REVIEWED custody event whose own ContentHash matches the manifest's *current* SHA256 -- the mere *existence* of a HASHED event for this EvidenceID is insufficient. cre.AuthorizeGrounded requires every

cited evidence ID to resolve to a manifest whose hash independently re-verifies via VerifyManifestHash, not merely a Finding.Hash that is internally self-consistent.

Proven by: TestAdvanceRefusesHashedEventBoundToDifferentContent, TestAdvanceRefusesReviewedEventBoundToDifferentContent, TestVerifyManifestHashDetectsTampering, the AuthorizeGrounded ErrEvidenceNotGrounded test suite (pkg/insurance/cre).

INV-005 -- EVERY EVIDENCE PREREQUISITE MUST BE IDENTITY-BOUND TO THE SAME EVIDENCE CONTENT.

Mechanism: the same hasCustodyActionBoundToContent check as INV-004, plus CustodyEvent.ContentHash itself being covered by the custody hash chain (so a forged binding cannot be inserted without detection).

Proven by: TestCustodyEventContentHashIsHashChainCovered, TestAdvanceRefusesHashedEventWithNoContentHashAtAll, TestReorderedCustodyChainFailsVerification.

INV-006 -- SERIALIZATION CANNOT MANUFACTURE AUTHORITY.

Mechanism: RegisterDraft and Submit reset authority-bearing fields on *any* input, including a value that arrived via json.Unmarshal -- a deserializer sets exported fields exactly the way a struct literal does, and neither construction path distinguishes the two.

Proven by: TestJSONDeserializedRecordCannotManufactureAuthority, TestJSONDeserializedManifestCannotManufactureAuthority, TestForgedSnapshotStateFieldCannotBeRestored (this round -- the direct "forged snapshot" framing: an honest JSON snapshot is intercepted and its state/manifest_hash fields rewritten, then shown to independently fail VerifyManifestHash AND to be forced back to DRAFT by RegisterDraft regardless).

INV-007 -- REPLAY CANNOT MANUFACTURE AUTHORITY.

Mechanism: transitionPrerequisiteLocked's gate is evaluated identically on every call to Advance, whether that call is the first time an event happens or a replayed reconstruction of history -- there is no "replay mode" flag or alternate code path anywhere in Registry that skips prerequisite checking.

Proven by: TestReplayOmittingReviewedEventCannotReachFinalized (this round -- a replay log that drops the REVIEWED event is refused ErrTransitionPrerequisiteNotMet and permanently cannot reach FINALIZED, confirmed by retry), TestConcurrentReplayDuringLiveMutationIsUnaffected (this round -- an independent replay concurrent with live mutation still converges on the exact same authoritative result).

INV-008 -- SNAPSHOT RESTORATION CANNOT MANUFACTURE AUTHORITY.

Mechanism: Registry has no Restore/Import/LoadSnapshot API anywhere. RegisterDraft is the *only* function that accepts an arbitrary Manifest value from a caller, and it unconditionally forces State=DRAFT and clears the hash fields regardless of what that value claims.

Proven by: TestForgedSnapshotStateFieldCannotBeRestored (this round), plus the repository-wide grep from the prior round confirming pkg/storage/snapshot and pkg/replay import neither pkg/evidence/manifest nor pkg/insurance/evidence (and vice versa) -- there is no live snapshot-restore code path touching these types at all today; see Part D for the honest scope note this implies.

INV-009 -- REPLICATION CANNOT MANUFACTURE AUTHORITY.

Mechanism: every hash this repository computes over authoritative content (computeManifestHash, computeCustodyHash, jcs.MustHash in cre.Authorize) is a pure function of semantic content -- no time.Now(), no randomness, no machine-local state. Two independent Registry instances ("Node A", "Node B") driven through the identical sequence of real, gated calls must therefore compute byte-identical authoritative state, which is exactly what replication correctness requires.

Proven by: TestReplayReproducesIdenticalFinalizedState (this round -- two independent registries converge on identical ManifestHash and CustodyChainHead, and both independently re-verify).

INV-010 -- AUTHORITY CANNOT BE SELF-BOOTSTRAPPED THROUGH A DOWNSTREAM GATE.

Mechanism: provenance.Registry.GrantTrust is the **only** function that can ever set Entry.TrustGranted = true; Register unconditionally forces TrustGranted=false regardless of caller input. No downstream consumer of trust (SetRights, and by extension cre.Authorize's own hypothesis/evidence checks) has any code path that grants trust itself -- each only **checks** trust that was already, separately, attributably granted.

Proven by: TestRegisterNeverAutoGrantsTrustRegardlessOfCallerInput, TestNamedNeverAutoTrustSourcesStayUntrustedUntilExplicitGrant, TestSetRightsRefusesUntrustedAuthority, TestConcurrentSetRightsChangesConvergeToASingleAuthorizedValue (this round -- proves the untrusted-caller refusal holds even when racing against legitimate concurrent authorized writers, not just sequentially).

PART C -- WHAT THIS ROUND FOUND AND CLOSED

C.1 NEW FINDING: CAUSATION.HYPOTHESISSET.ADD DID NOT RESET A CALLER-SUPPLIED STATUS

Writing INV-001 as a literal, checkable claim ("no caller may directly construct an authoritative state") required checking it against every subsystem the model in Part A maps -- not just the three already covered by the prior four rounds. Doing so surfaced a real, previously unaudited gap: HypothesisSet.Add only defaulted Status when the caller left it blank (if h.Status == "" { h.Status = StatusUnproven }) -- a caller supplying Hypothesis{Status: StatusSupported, SupportingEvidence: []string{"EV-forged-1"}} had that value accepted verbatim, bypassing computeStatus entirely. This is the exact shape of bypass the Final Authority Hardening Round found and fixed in evidence.Registry.Submit.

A repository-wide grep confirmed zero production callers construct a causation.Hypothesis{} literal or call Add with a non-default

Status anywhere outside tests today -- this is a structural gap, not a live exploit, exactly the same honest category as the GrantTrust root-of-trust audit and the snapshot/replay absence audit from prior rounds. Consistent with this engagement's standing discipline of closing such gaps defensively rather than waiting for a caller to find them, Add now unconditionally forces Status = StatusUnproven on every input, matching RegisterDraft/Submit's own pattern exactly.

Regression proof (break-test-fix-restore): the reset was temporarily narrowed back to the old empty-string-only default; TestHypothesisSetAddNeverTrustsACallerAssertedSupportedStatus failed with expected a caller-asserted StatusSupported to be discarded on Add (got "SUPPORTED"); the fix was restored and the test passes again. The pre-existing TestHypothesisSetAddRejectsDuplicateAndInvalid test, which had asserted the OLD (bypassable) behavior -- that an unrecognised Status value was rejected -- was updated to assert the new, correct behavior: Add always succeeds and the stored Hypothesis always reads StatusUnproven, regardless of what the caller supplied.

C.2 DISTRIBUTED REPLAY / SNAPSHOT-FORGERY AUDIT (TASK CLOSING INV-007/008/009)

The reviewer named three concrete proofs. Each is answered honestly against what this codebase actually contains:

1. "Node A -> FINALIZED Evidence -> snapshot -> Node B restore must produce exactly the same authority state." No live snapshot/distributed-restore mechanism touches manifest.Manifest or evidence.Record in this repository today (confirmed by repository-wide grep: pkg/storage/snapshot and pkg/replay import neither package, and vice versa) -- there is nothing live to run this exact scenario against. The honest, code-grounded substitute is TestReplayReproducesIdenticalFinalizedState: two fully independent Registry instances, driven through the identical sequence of real, gated calls, are proven to compute byte-identical ManifestHash and CustodyChainHead, and both independently re-verify -- which is exactly what "the same authority state" means for a hash-addressed system, since the hash IS the authority state's fingerprint.
2. "Snapshot with forged status must be rejected." TestForgedSnapshotStateFieldCannotBeRestored marshals a genuine StateDraft manifest to JSON, rewrites the intercepted bytes to claim FINALIZED with a fabricated hash, and proves the forgery is inert two independent ways: VerifyManifestHash rejects the fabricated hash, AND there is no Restore/Import API on Registry at all -- RegisterDraft is the only construction path from an arbitrary value, and it forces State=DRAFT regardless.
3. "Replay omits REVIEWED event must not result in FINALIZED." TestReplayOmittingReviewedEventCannotReachFinalized faithfully replays every real event except REVIEWED and proves both the immediate refusal (ErrTransitionPrerequisiteNotMet) and that the gap is permanent -- retrying Advance(..., StateFinalized, ...) still fails, and Latest() never reports StateFinalized.

C.3 CONCURRENCY SEMANTIC-CORRECTNESS AUDIT (TASK CLOSING THE "RACE-FREE != STATE-MACHINE-SAFE" ASK)

Seven new tests, one per named scenario, all passing under -race and stable across 20 repeated runs:

Named scenario	Test	Semantic guarantee proven
----------------	------	---------------------------

--- --- ---

double finalize TestConcurrentDoubleFinalizeHasExactlyOneWinner of N concurrent
finalize() calls, exactly one succeeds; the manifest ends FINALIZED with a hash that
independently verifies

finalize + mutate TestConcurrentFinalizeAndCustodyMutateNeverStalesTheHash 50
concurrent legitimate custody mutations racing a finalize() never stale ManifestHash; the
append-only custody log still independently verifies

finalize + supersede
TestConcurrentFinalizeAndSupersedeNeverSupersedesAnUnfinalizedParent supersede() never
succeeds against a not-yet-finalized parent no matter the race outcome; a losing supersede()
succeeds legitimately on retry; the original FINALIZED hash survives the race unchanged

supersede + submit
TestConcurrentMarkSupersededAndDuplicateSubmitNeverResurrectsEvidence 20 concurrent
duplicate Submit attempts against a record concurrently being superseded are all refused
(ErrDuplicateEvidence); the supersession's own fields are never torn or overwritten by a
losing submit

concurrent rights change
TestConcurrentSetRightsChangesConvergeToASingleAuthorizedValue two authorized authorities
racing different target rights states always converge to exactly one of the two legitimate
values; an untrusted third caller racing the same operation is refused every time,
regardless of interleaving

duplicate transition TestConcurrentDuplicateAdvanceTransitionHasExactlyOneWinner of
20 concurrent identical DRAFT->INGESTED transitions, exactly one wins; the rest observe the
post-transition state and are refused ErrInvalidTransition, never silently re-applied

replay during mutation TestConcurrentReplayDuringLiveMutationIsUnaffected an
independent replay running concurrently with live post-finalization mutation of the original
still reproduces the exact original ManifestHash; neither interferes with the other

This directly answers the reviewer's own framing: go test -race
already proved no *unsynchronized* memory access occurs (Registry
serializes every method via its own mutex) -- these seven tests prove the
semantic outcome of contested transitions is always correct, not
merely that the contest itself is memory-safe.

PART D -- RECALIBRATED STATUS VS. THE REVIEWER'S OWN TABLE

Layer	Reviewer's status	This round's honest status
--- --- ---		
Evidence authority (Level 2)	Engineering hardened	Unchanged: still Engineering hardened
Manifest authority (Level 2)	Engineering hardened	Unchanged: still Engineering hardened
Finding authority (Level 2)	Engineering hardened	Unchanged: still Engineering hardened
Hypothesis authority (C.1) -- now Engineering hardened on the same basis as the other three, not merely by omission of audit	Engineering hardened	One real gap found and closed this round
Trust root	No app self-bootstrap found / Deployment governance remains	Unchanged: GrantTrust's ultimate root is still an external policy/attestation document outside this repository's code -- that boundary is structural, not a gap this repository's code can close
Serialization authority	Major bypass addressed	Unchanged, now with the explicit "forged snapshot" framing proven (C.2.2)
Replay/replication authority	Needs final adversarial validation	Closed this round -- three tests directly answering the three named proofs (C.2), honestly scoped to what this codebase actually contains (no live snapshot/replay integration exists to test against a fortiori)
Concurrency/state-transition semantics	Needs validation	Closed this round -- seven

tests, one per named scenario, all passing under -race (C.3)

This remains Level 2 (Engineering Verified) work on this engagement's own 4-level maturity model. No claim of Level 3 (controlled external validation) or Level 4 (production proven) is made anywhere in this report. The Trust Authority Model in Part A is a specification grounded in, and traceable to, real code and real tests -- it is not a promise that a live distributed cluster has been run against it, because no such cluster exists in this repository to run.

VERIFICATION

gofmt -l .	clean
go build ./...	clean
go vet ./...	clean
go test ./...	full repository suite, all packages pass
go test -race (manifest, evidence, causation, and every other affected package)	clean, no data races

Package-level detail:

- pkg/evidence/manifest: 33/33 tests pass (3 replay/forgery + 5 concurrency tests new this round)
- pkg/insurance/evidence: 64/64 tests pass (2 concurrency tests new this round)
- pkg/insurance/causation: all tests pass (1 fixed test + 1 new adversarial test for the Add fix)

HONEST SCOPE BOUNDARY

- The Trust Authority Model (Part A) is a specification unifying four real, already-hardened subsystems -- it does not introduce a fifth, generic authority type, because no such type exists in the running code and inventing one here would not be truthful to what actually ships.
- The distributed-replay proofs (Part C.2) are the honest substitute for a live Node-A/Node-B snapshot-restore test: no such live integration exists in this repository between the storage/replay layer and the Evidence/Manifest authority layer, and this is stated plainly rather than padded with an inapplicable test.
- The concurrency proofs (Part C.3) demonstrate semantic correctness under real concurrent load against this repository's actual Registry types; they do not simulate network partitions, multi-process concurrency, or an actual distributed consensus protocol, none of which exist in this codebase to test against.
- The trust root (GrantTrust's ultimate authorization) remains, as every prior round has said, an external policy/attestation act outside this repository's code -- no amount of engineering work inside this repository closes that boundary, because it is not this repository's boundary to close.